String and Pattern Matching

String Matching

- To find the occurrence(s) of a substring (pattern PPP) in a larger string (text TTT).

Applications

Text editors (e.g., find and replace).

Plagiarism detection.

Computational biology (e.g., DNA sequence matching).

Network security (e.g., malware detection).

Brute Force

- Compares the pattern PP to the text TT character by character at every possible starting position
- Start at the first character of TT and align PP with it.
- Compare each character of PP with the corresponding character in TT.
- If all characters match, record the position.
- Shift PP one position to the right and repeat until the end of TT.

Brute Force

- **Algorithm Steps**
- Let **T** = text of length *n*
  Let **P** = pattern of length *m*
- For i from 0 to n–m:
- Compare T[i … i+m-1] with P[0 … m-1]
- If all characters match → pattern found.

Brute Force

Time Complexity: $O(m \cdot n)$, where $n =$ length of T and $m =$ length of P.

Space Complexity: $O(1)$.

Example:

$T =$ "$ABABABACD$", $P =$ "$ABAC$"

- Check positions $T[0:4]$, $T[1:5]$, $T[2:6]$, ..., until a match is found.

Brute Force

Dry Run
T = “ABCDEABCD”
P = “ABC”

```
i = 0: T[0..2] = ABC → match ✓
i = 1: T[1..3] = BCD → mismatch
i = 2: T[2..4] = CDE → mismatch
i = 3: T[3..5] = DEA → mismatch
i = 4: T[4..6] = EAB → mismatch
i = 5: T[5..7] = ABC → match ✓
```

Matches at indices: 0, 5

Rabin Karp

- The algorithm calculates a hash value for the pattern and for substrings of the text.
- Hashing allows us to compare these values instead of comparing the actual strings, speeding up the process.
- If the hash values match,
  - performs an additional character-by-character check
  - to confirm the match (to handle hash collisions).

Rolling Hash in Rabin Karp

- To calculate the hash of the next substring efficiently, Rabin-Karp uses a **rolling hash**:
- Subtract the hash contribution of the outgoing character.
- Add the hash contribution of the incoming character.

Rabin Karp Algorithm

- Calculate the hash of the pattern (H(P)H(P)).
- Calculate the hash of the first substring of the text (H(T[0:m])H(T[0:m]), where mm is the length of the pattern).
- Slide the window across the text:
- Update the hash value for the next substring using the rolling hash formula.
- If the hash matches H(P)H(P), verify the substring by comparing characters.

- Repeat until the end of the text.

Rabin Karp Example

$T = \text{"}31415926535\text{"}, P = \text{"}159\text{"}$

- Compute $H(P) = H(\text{"}159\text{"}) = 1 + 5 + 9 = 15.$
- Calculate rolling hashes for substrings $T[0:3], T[1:4], \ldots.$

Rabin Karp Example

Step 1: Initialize

- **Text (T):** `31415926535`
- **Pattern (P):** `159`
- Length of the pattern (m): 3

Step 2: Hash Function

For simplicity, let's use a **basic hash function**: $H(S) = \text{Sum of ASCII values of characters in } S$

- Hash of the pattern:

$$H(P) = H(\text{"159"}) = 1 + 5 + 9 = 15$$

- Hash of the first substring of the text:

$$H(T[0:3]) = H(\text{"314"}) = 3 + 1 + 4 = 8$$

Step 3: Slide and Compare

1. **First Window:** $T[0:3] = \text{"314"}$, $H(\text{"314"}) = 8$
   - $H(T[0:3]) \neq H(P)$, no match.
2. **Second Window:** $T[1:4] = \text{"141"}$
   - Update hash:
     $$H(T[1:4]) = H(T[0:3]) - \text{ASCII}(T[0]) + \text{ASCII}(T[3])$$
     $$H(\text{"141"}) = 8 - 3 + 1 = 6$$
   - $H(\text{"141"}) \neq H(P)$, no match.
3. **Third Window:** $T[2:5] = \text{"415"}$
   - Update hash:
     $$H(\text{"415"}) = 6 - 1 + 5 = 10$$
   - $H(\text{"415"}) \neq H(P)$, no match.
4. **Fourth Window:** $T[3:6] = \text{"159"}$
   - Update hash:
     $$H(\text{"159"}) = 10 - 4 + 9 = 15$$
   - $H(\text{"159"}) = H(P)$, potential match!
   - Verify characters: `"159" == "159"`, so a match is found at index 3.

Step 4: Continue Sliding

Continue sliding the window and updating the hash until the end of the text.

Advantages of Rabin Karp

- Efficient for finding multiple matches.
- Rolling hash reduces computational effort.

Rabin Karp - Disadvantages

- Hash collisions can increase runtime due to character-by-character verification.
- Not always the fastest for single-pattern matches compared to KMP.

Algorithm	Best Use Case	Time Complexity	Space Complexity
Brute Force	Small texts and patterns.	$O(m \cdot n)$	$O(1)$
Rabin-Karp	Multiple pattern matching.	$O(m + n)$ (average)	$O(1)$
KMP	Large texts, frequent patterns.	$O(m + n)$	$O(m)$

Knuth-Morris-Pratt (KMP)

- KMP uses the **structure of the pattern** to avoid redundant comparisons.
- Preprocesses the **pattern** string and creates an array called the **Longest Prefix Suffix** (lps) array which indicates how much of the pattern can be reused after a mismatch.

- LPS is the **Longest Proper Prefix** which is also a **Suffix**

Longest Proper Prefix

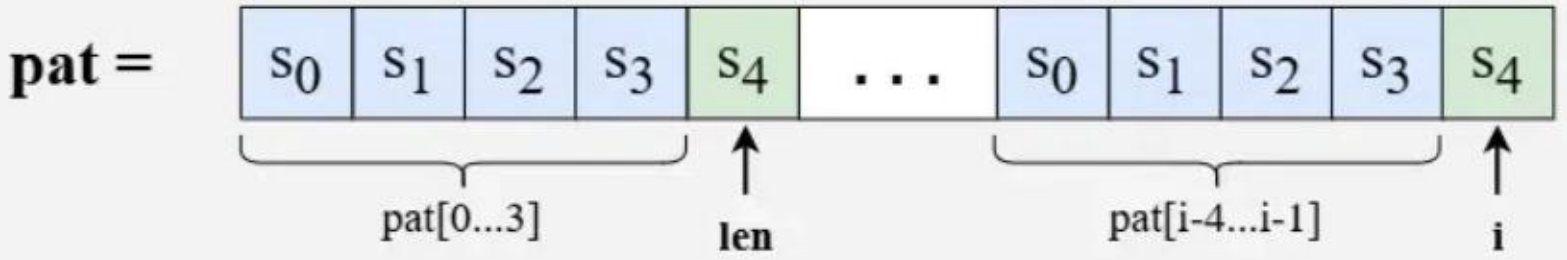
- is a prefix that doesn't include **whole** string. For example,
- prefixes of "abc" are "", "a", "ab" and "abc" but proper prefixes are "", "a" and "ab" only.
- Suffixes of the string are "", "c", "bc", and "abc"

Constructing LPS sub array

Case 01

Case 01:
pat[i] = pat[len]

Assume we are at index i and len = 4, so this means that pat[i-4...i-1] matches with pat[0...3].
Now, if pat[i] = pat[len] then we can simply increment len by 1, so len = 5 and update lps[i] = 5.

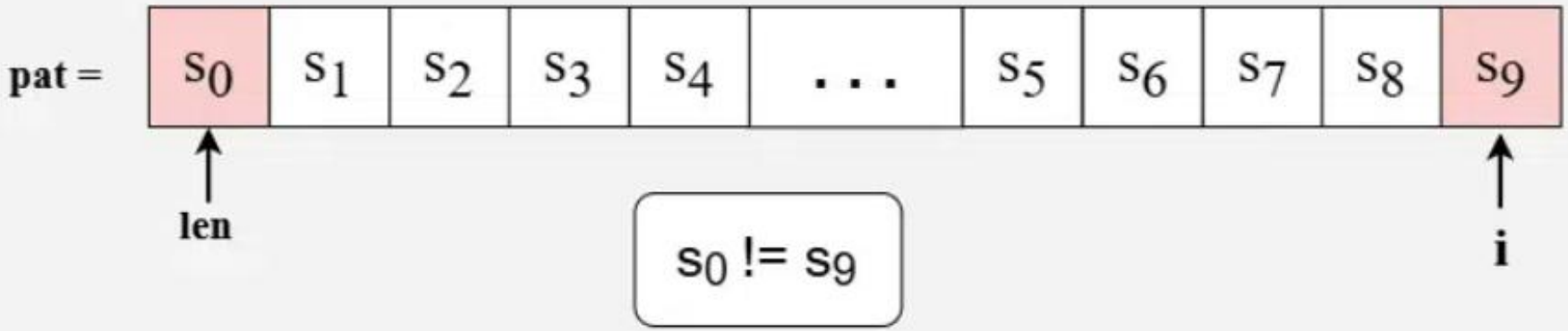


Different Cases for Construction of lps array

Case 02

Case 02:
pat[i] != pat[len] and len = 0

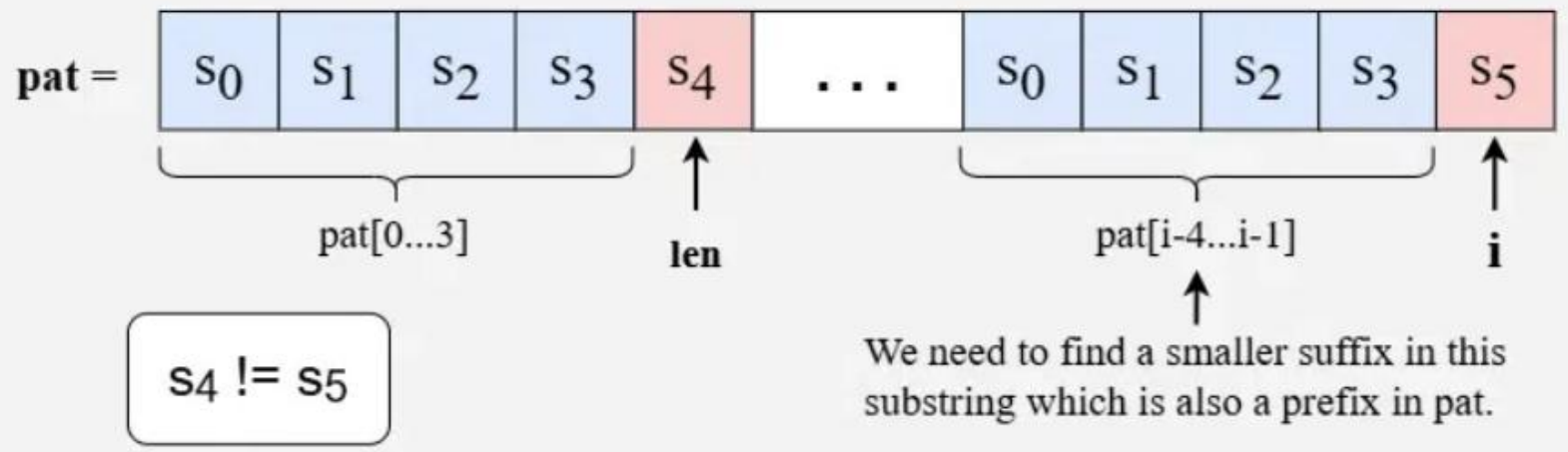
Assume we are at index i and len = 0, so this means that there were no matching characters earlier and the current characters are also not matching, so lps[i] = 0.



Case 03

Case 03:
pat[i] != pat[len] and len > 0

Assume we are at index i and len = 4, so this means that pat[i-4...i-1] matches with pat[0...3]. Now, if pat[i] != pat[len] then we can't extend the LPS at index i - 1, so we need a smaller suffix of **pat[i-len...i-1]** that is also a proper prefix of **pat**.

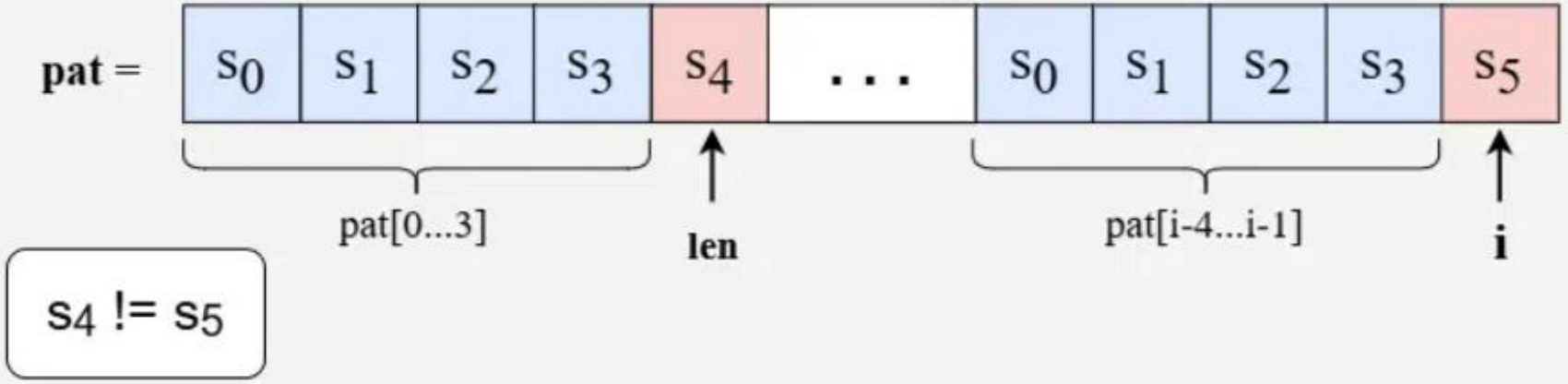


Different Cases for Construction of lps array

Case 03

Case 03 (continued):
pat[i] != pat[len] and len > 0

Since pat[0...3] = pat[i-4...i-1], so instead of searching in pat[i-4...i-1], we can find the longest suffix which is also a prefix in pat[0...3]. To find the longest suffix which is also a prefix in pat[0...3], we can check lps[len-1]



$s_4 != s_5$

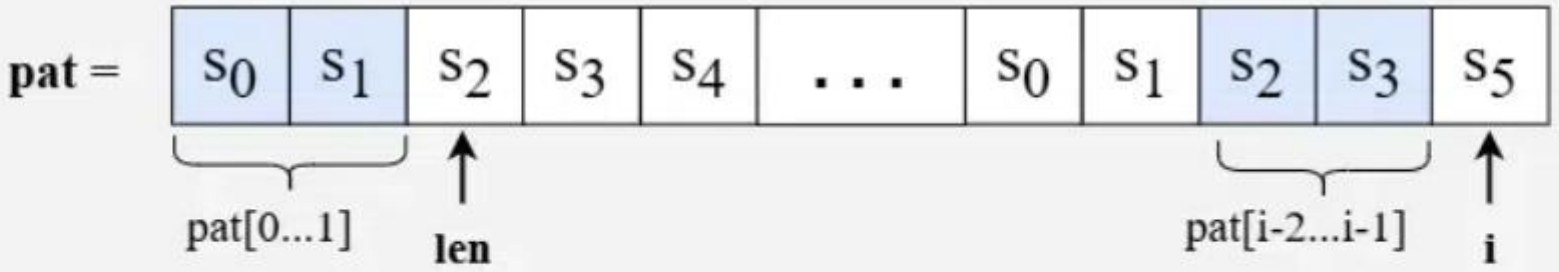
Case 03

Case 03 (continued):
pat[i] != pat[len] and len > 0

Say if lps[len - 1] is 2, then it means that $s_0s_1 = s_2s_3$.

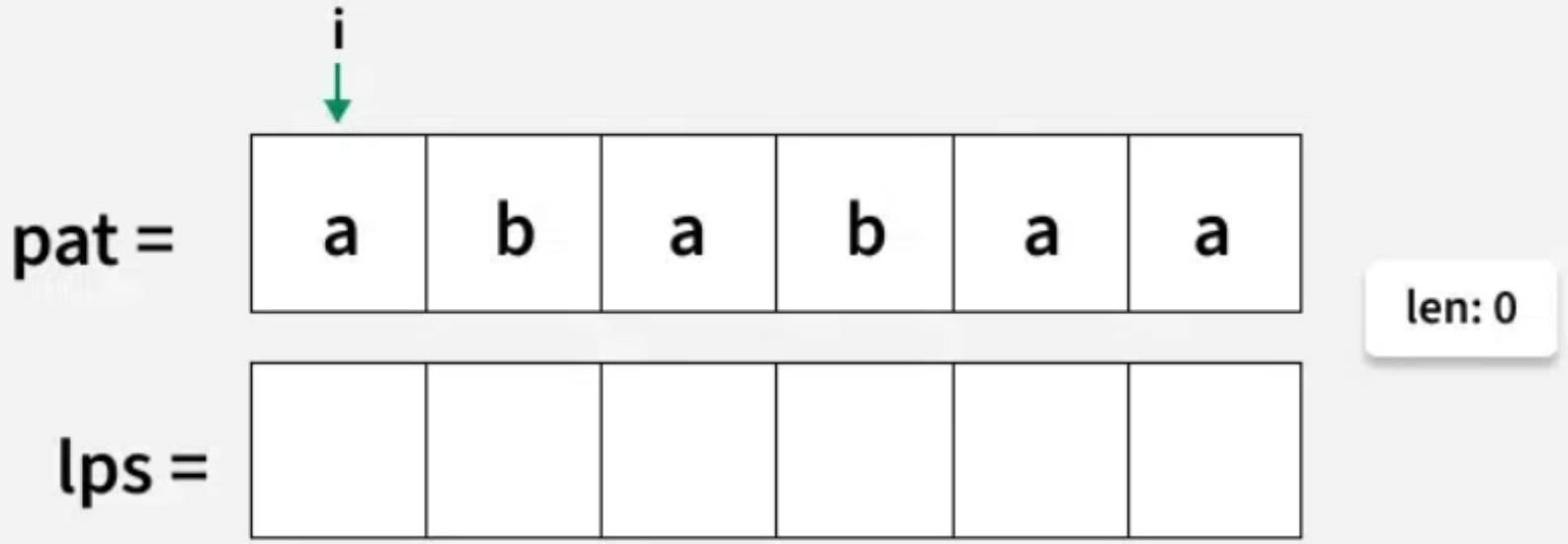
So, update len = lps[len-1] = 2.

Now, we will check if pat[len] is equal pat[i] to find a smaller suffix ending at i which is also a proper prefix.



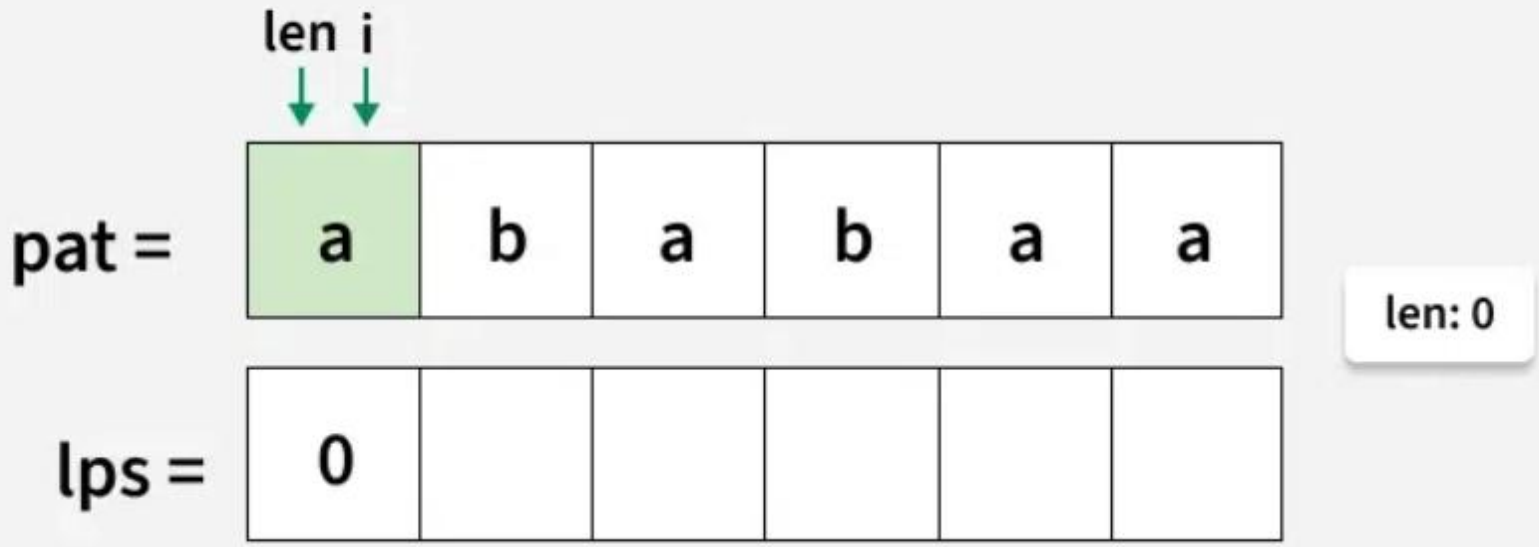
Example Construction

01 Step | Initialize len to zero and create an lps[] array of the same length as pattern. Place a pointer i at the start of the pattern for traversal.

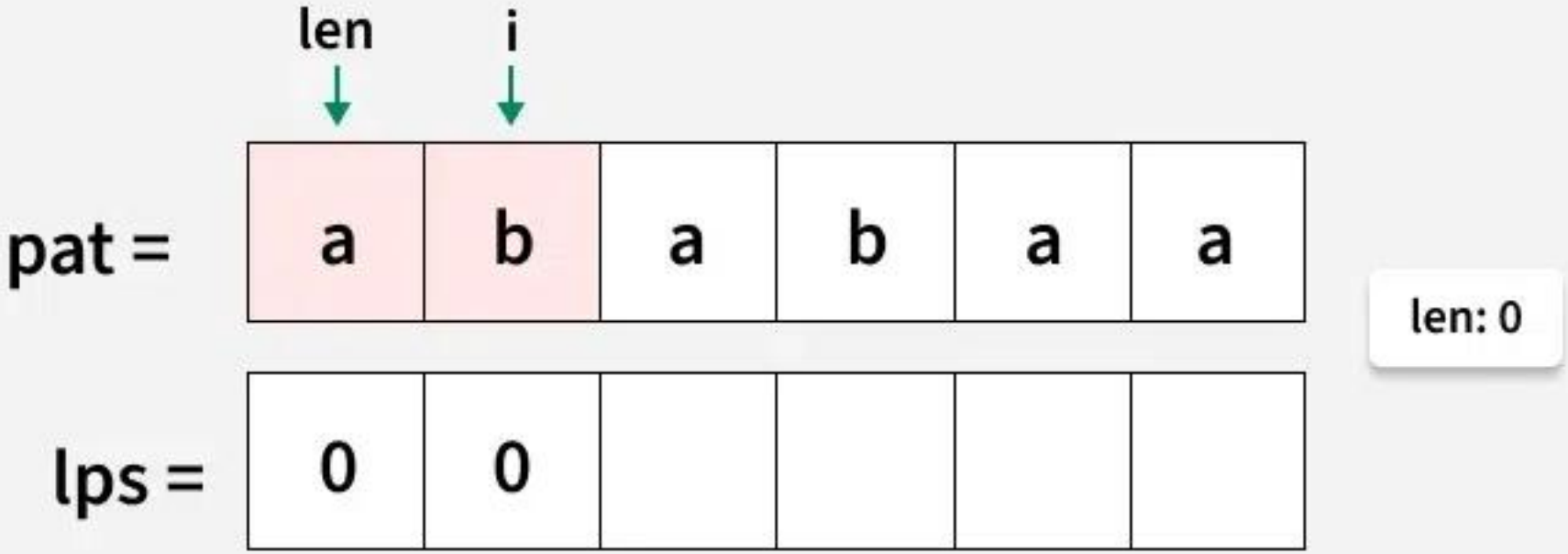


02 Step

lps[0] is always 0 because there is no proper prefix for a single character, move i to next index.

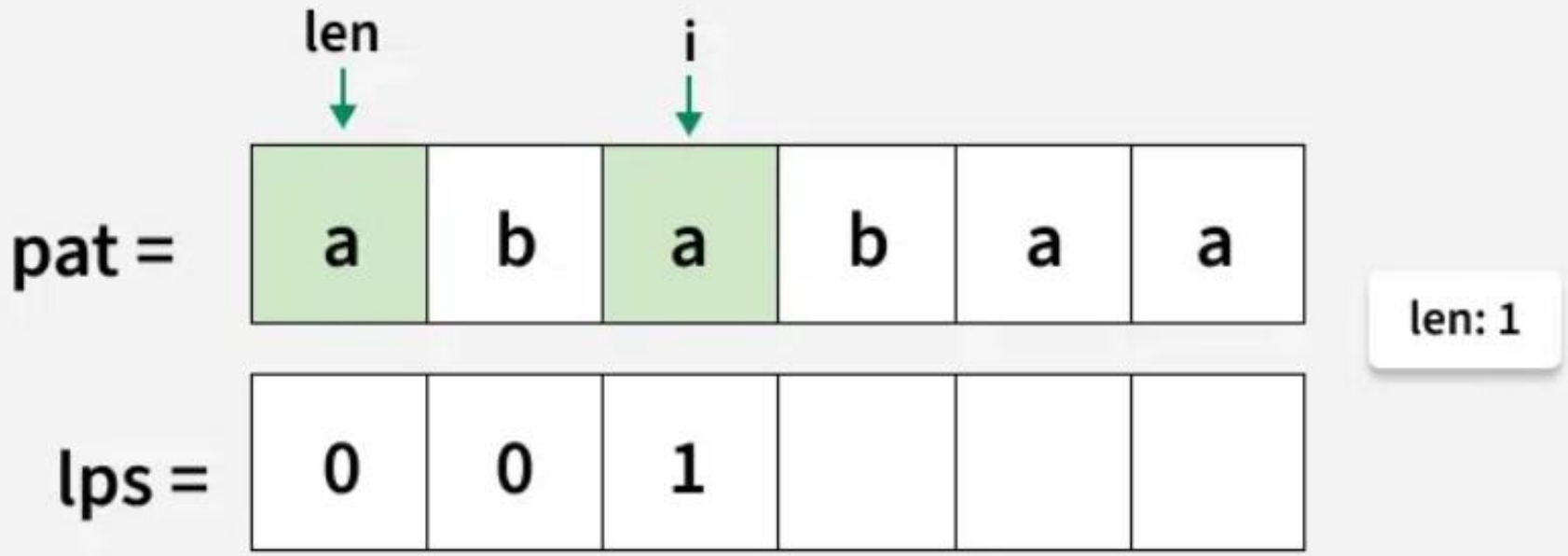


03 Step | Since pat[len] and pat[i] do not match and len is 0, so lps[i] will be zero. Move i to the next index.

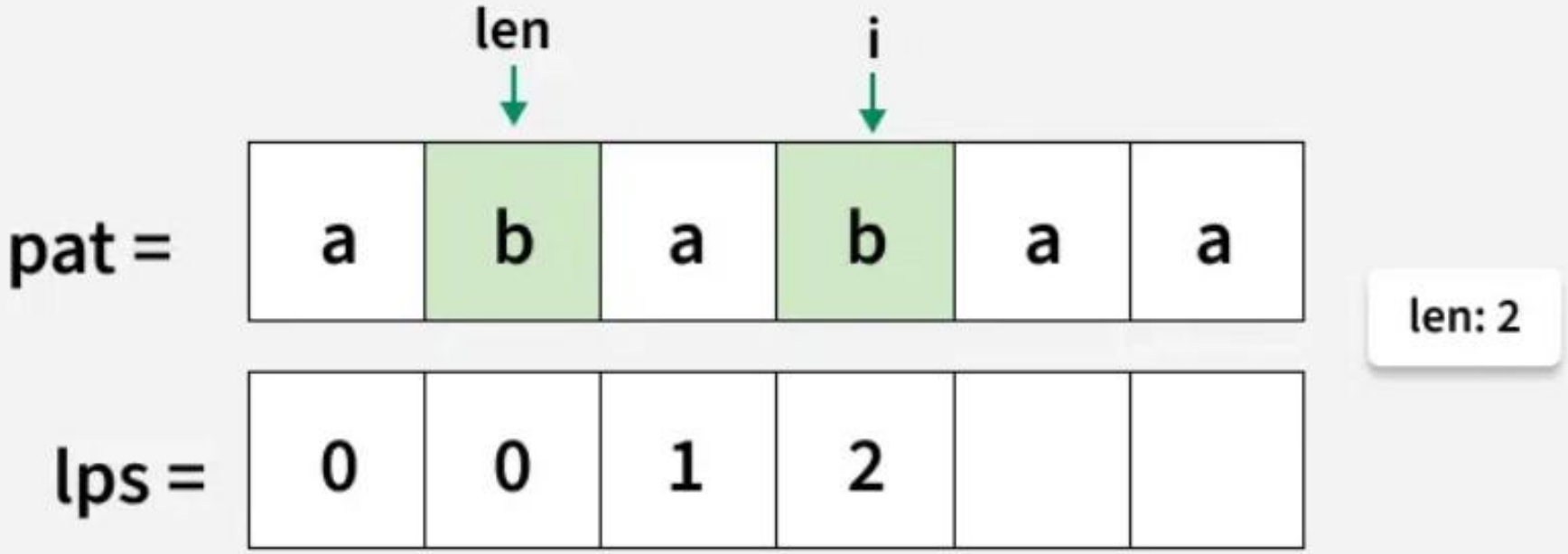


Construction of lps array

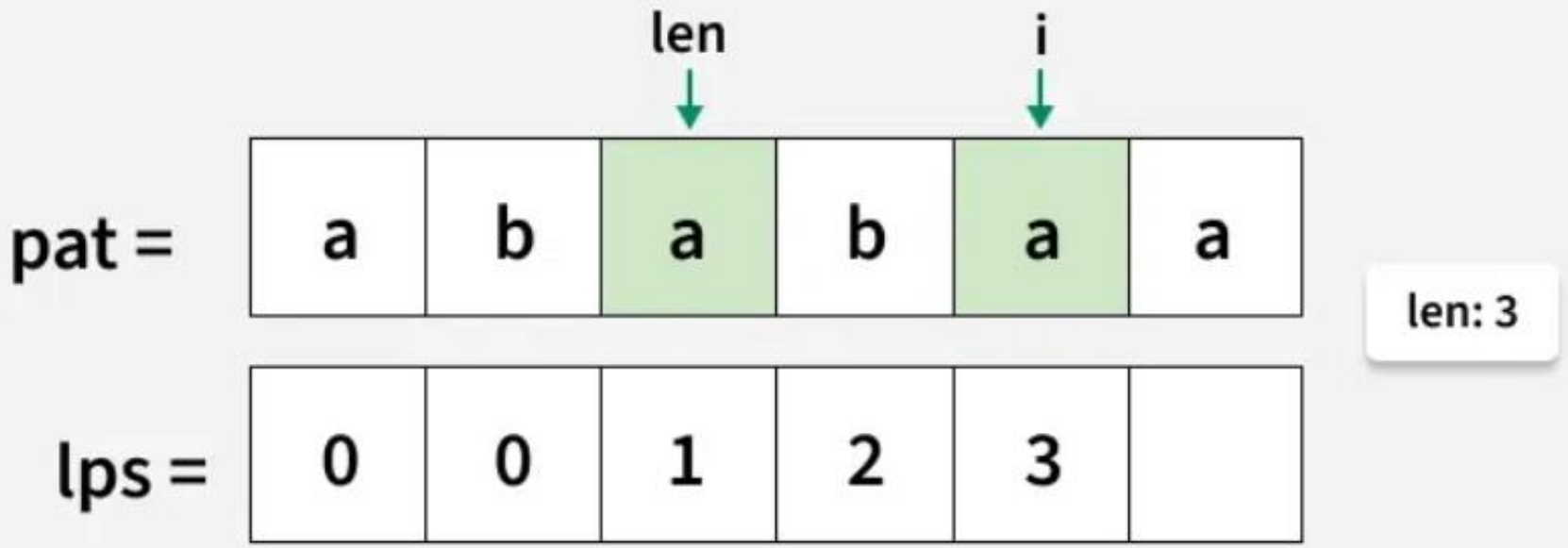
04 Step | Since pat[len] and pat[i] match, increment len and store it in lps[i].
Move i to the next index.



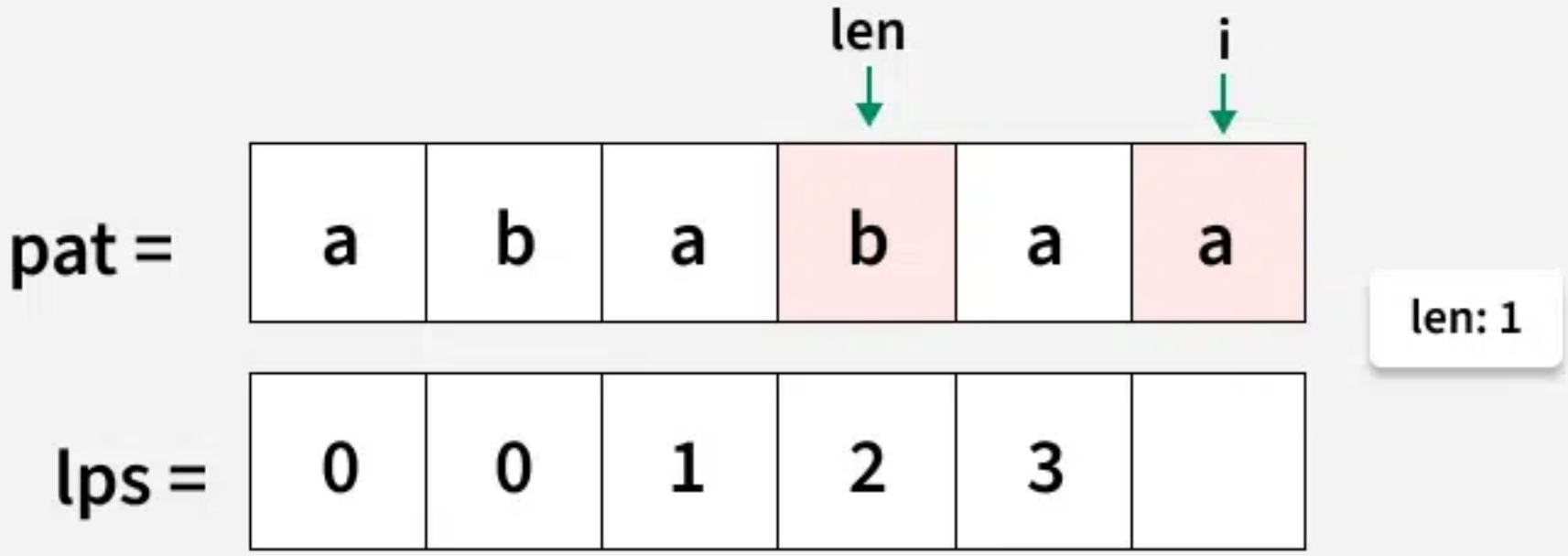
05 Step | Since pat[len] and pat[i] match, increment len and store it in lps[i]. Move i to the next index.



06 Step | Since pat[len] and pat[i] match, increment len and store it in lps[i]. Move i to the next index.

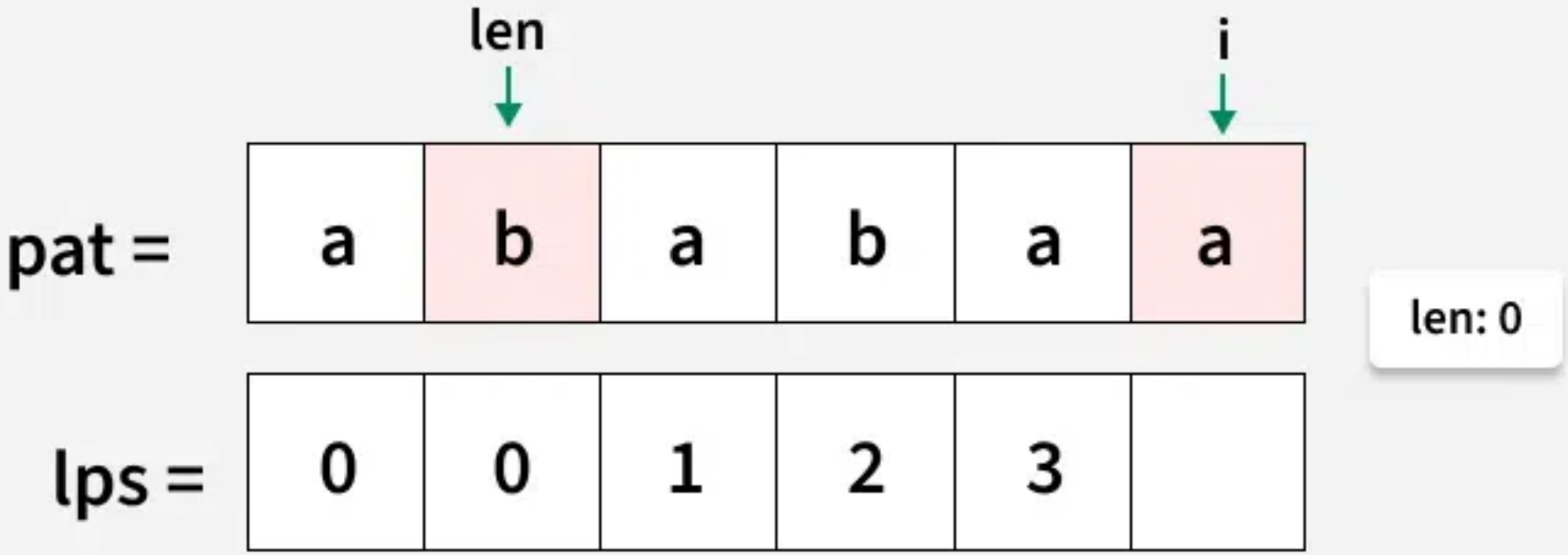


07 Step | Since pat[len] and pat[i] do not match, set len to lps[len-1] = lps[2] = 1.



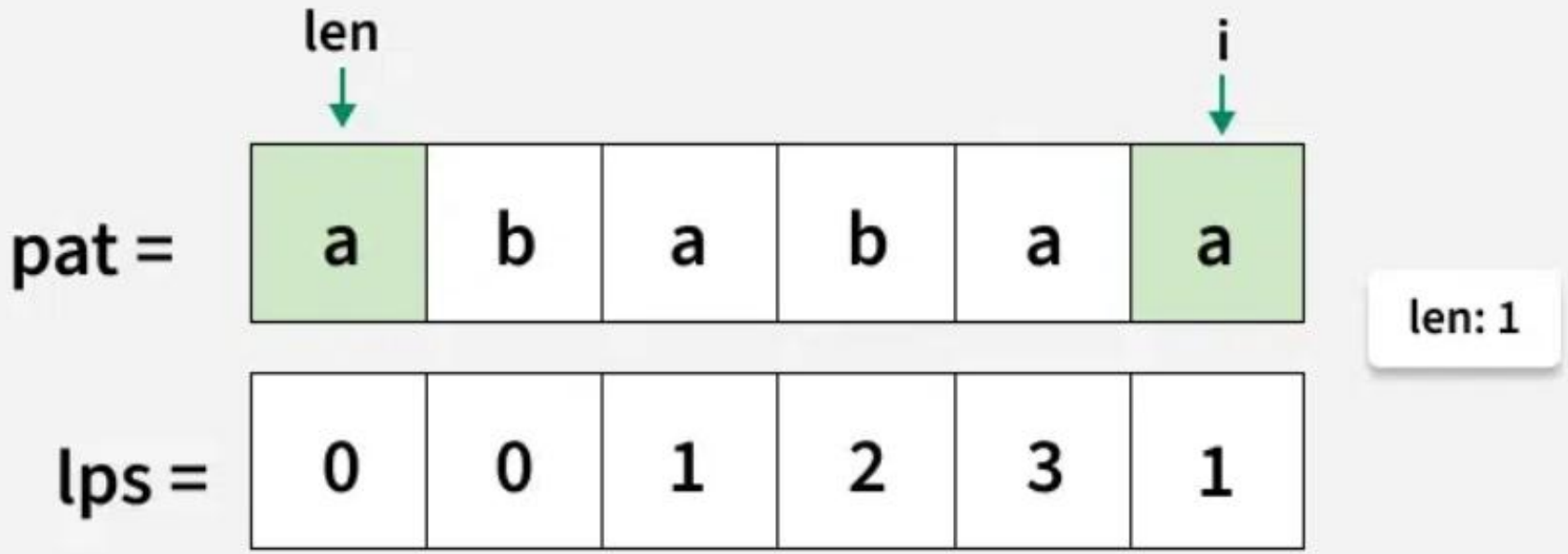
Construction of lps array

08 Step | Since pat[len] and pat[i] do not match set len to lps[len-1] = lps[0] = 0.



Construction of lps array

09 Step | Since pat[len] and pat[i] match increment len to 1, and store it in lps[i].



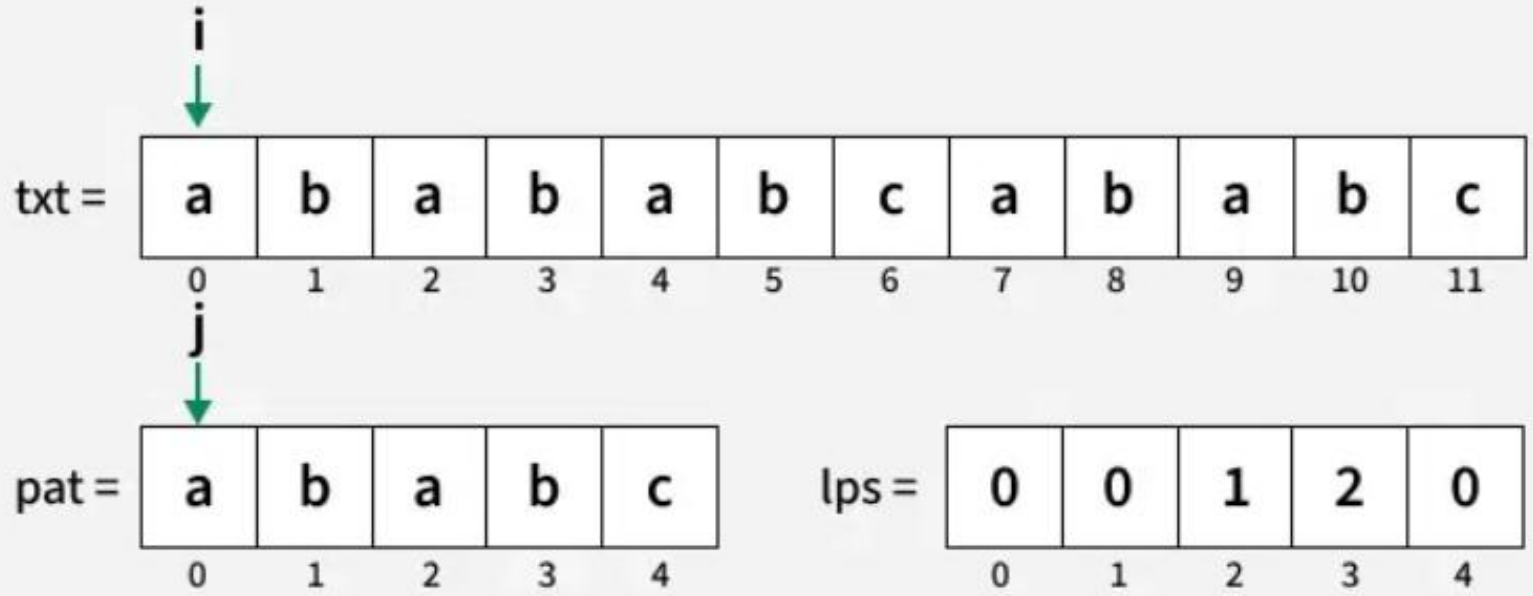
Implementation of the Algorithm

- We initialize two pointers
  - one for the text string and
  - another for the pattern.
- When the characters at both pointers match,
  - we increment both pointers and continue the comparison.
- If they do not match
  - we reset the pattern pointer to the last value from the LPS array because that portion of the pattern has already been matched with the text string.
- Similarly, if we have traversed the entire pattern string,
  - we add the starting index of occurrence of pattern in text, to the result and continue the search from the lps value of last element of the pattern.

Example

01 Step

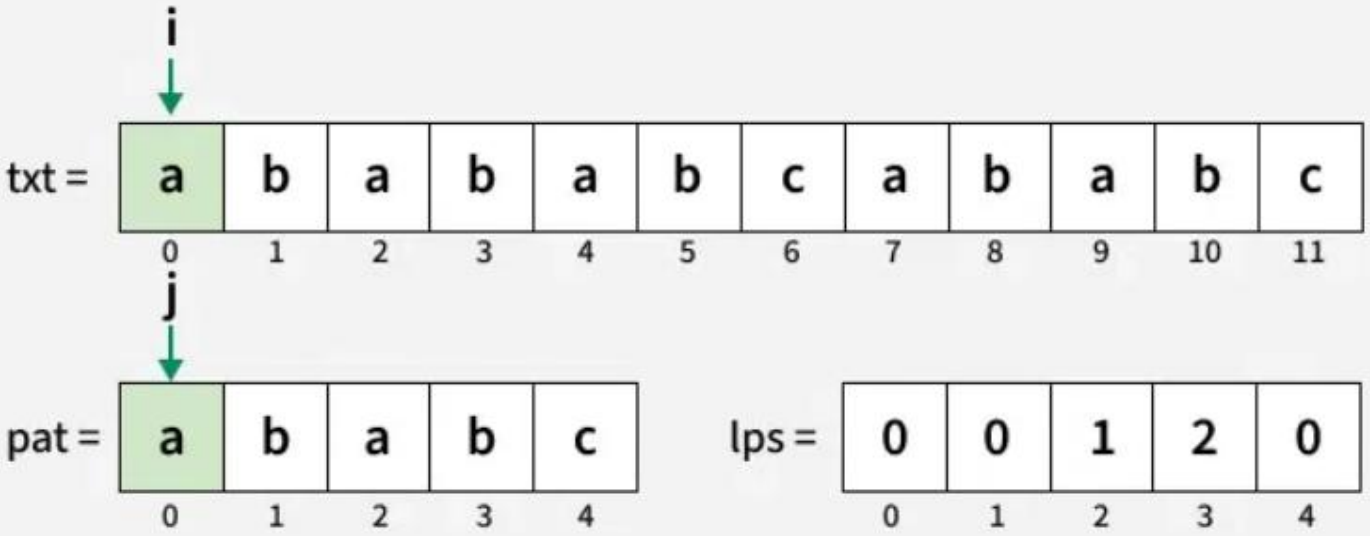
Initialize pointers at the beginning of both text and pattern.
The lps for the given pattern would be lps[] = {0, 0, 1, 2, 0}



Result = []

KMP Algorithm for Pattern Searching

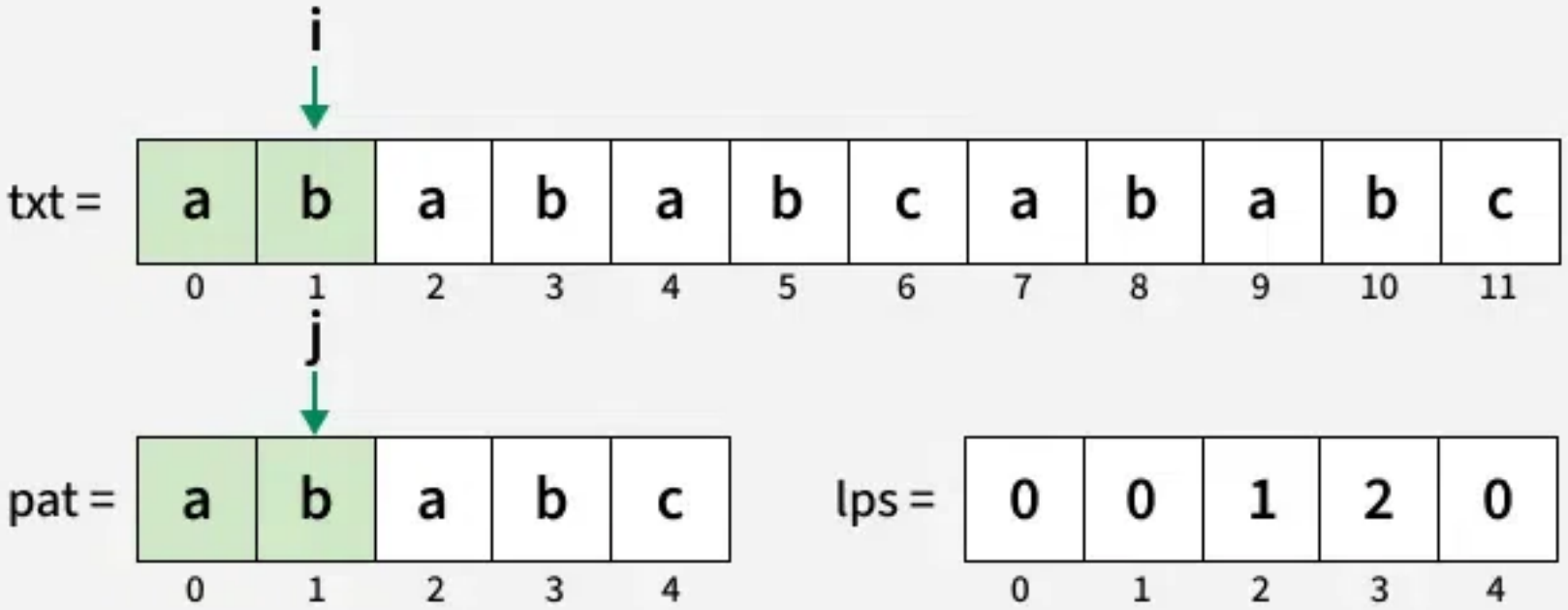
02 Step | Since txt[i] and pat[j] match, increment both i and j.



Result = []

KMP Algorithm for Pattern Searching

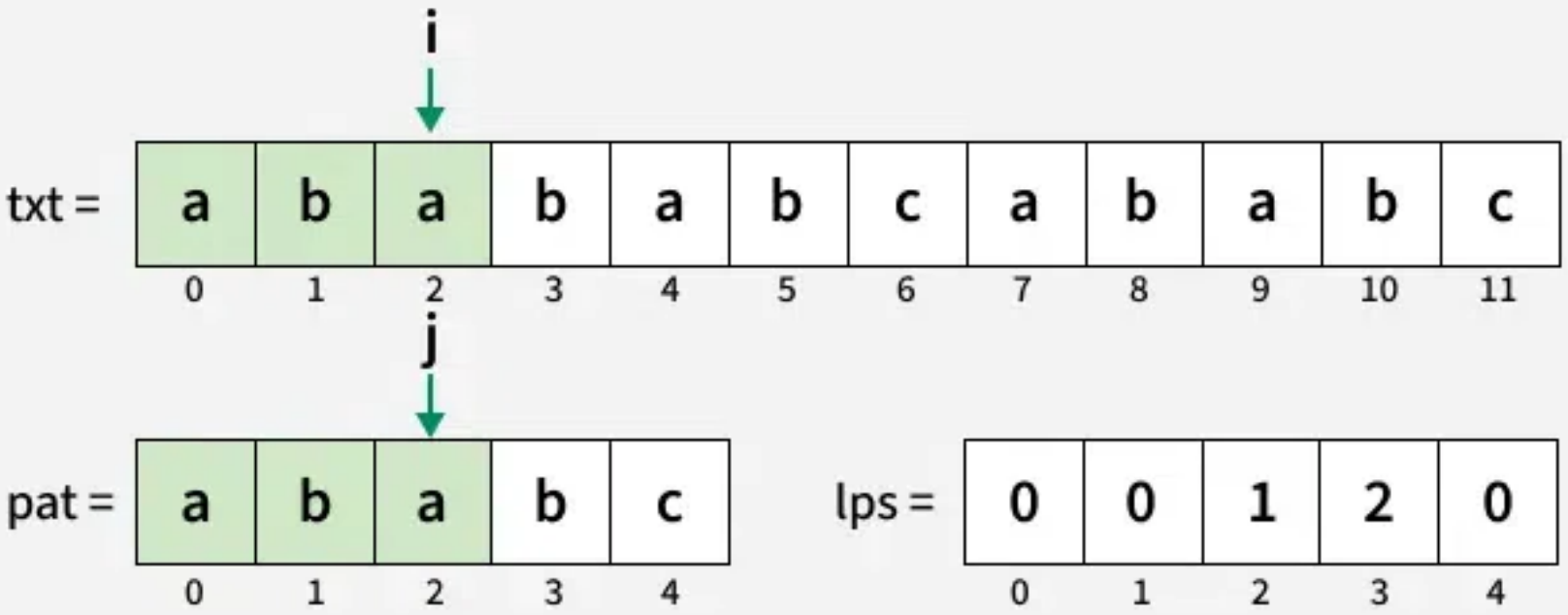
03 Step | Since txt[i] and pat[j] match, increment both i and j.



Result = []

KMP Algorithm for Pattern Searching

04 Step | Since txt[i] and pat[j] match, increment both i and j.

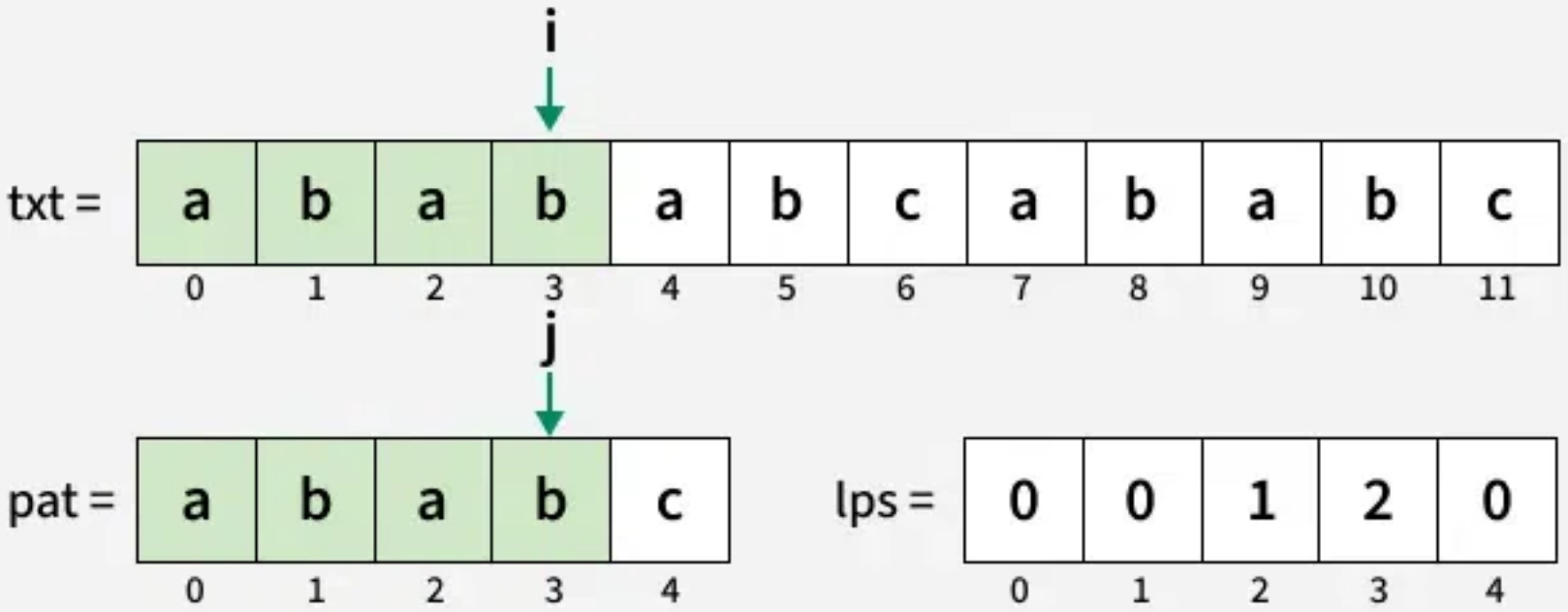


Result = []

KMP Algorithm for Pattern Searching

05 Step

Since txt[i] and pat[j] match, increment both i and j.

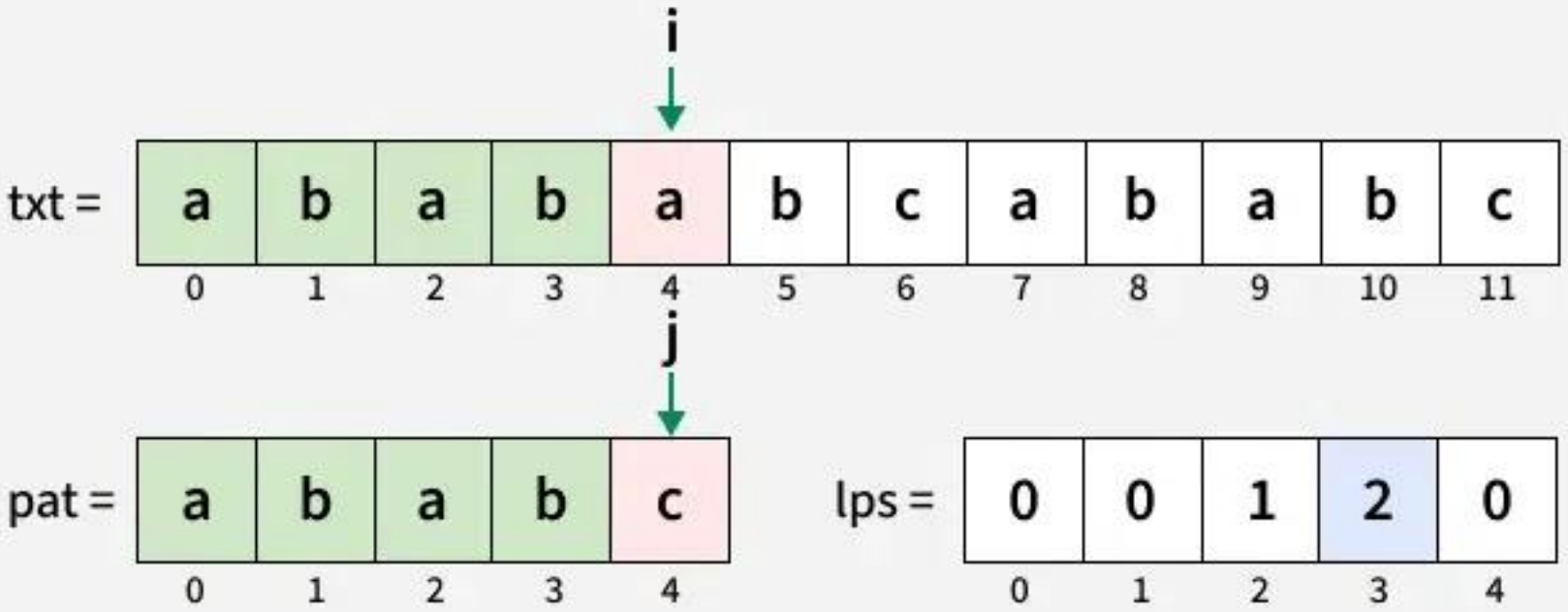


Result = []

KMP Algorithm for Pattern Searching

06 Step

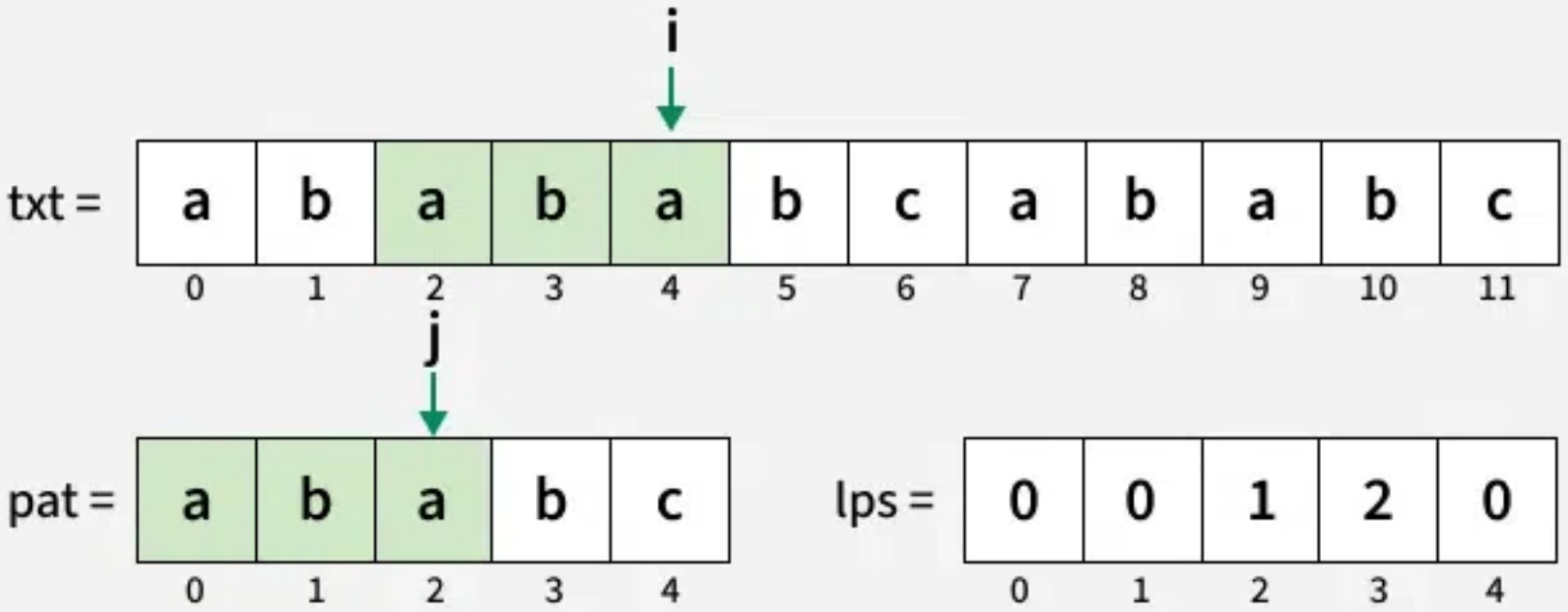
Since txt[i] and pat[j] do not match, update j to lps[j-1] = 2



Result = []

KMP Algorithm for Pattern Searching

07 Step | Since txt[i] and pat[j] match, increment both i and j.

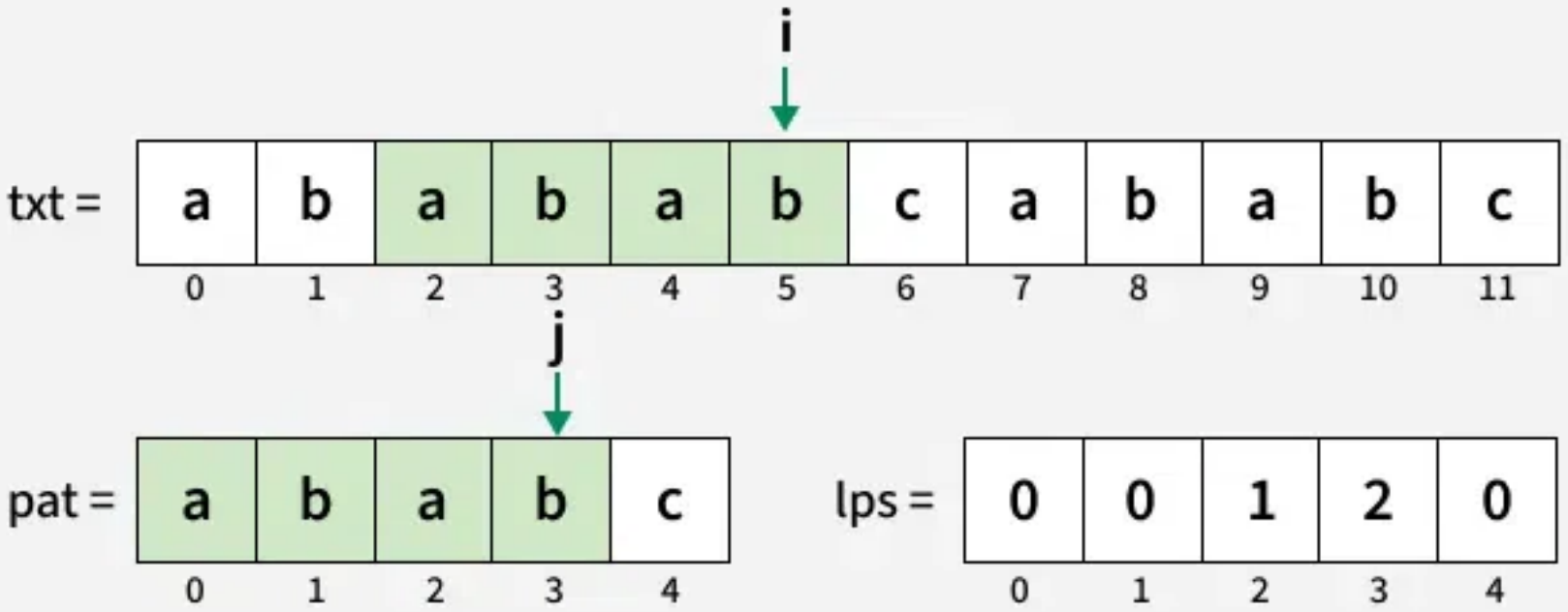


Result = []

KMP Algorithm for Pattern Searching

08 Step

Since txt[i] and pat[j] match, increment both i and j.



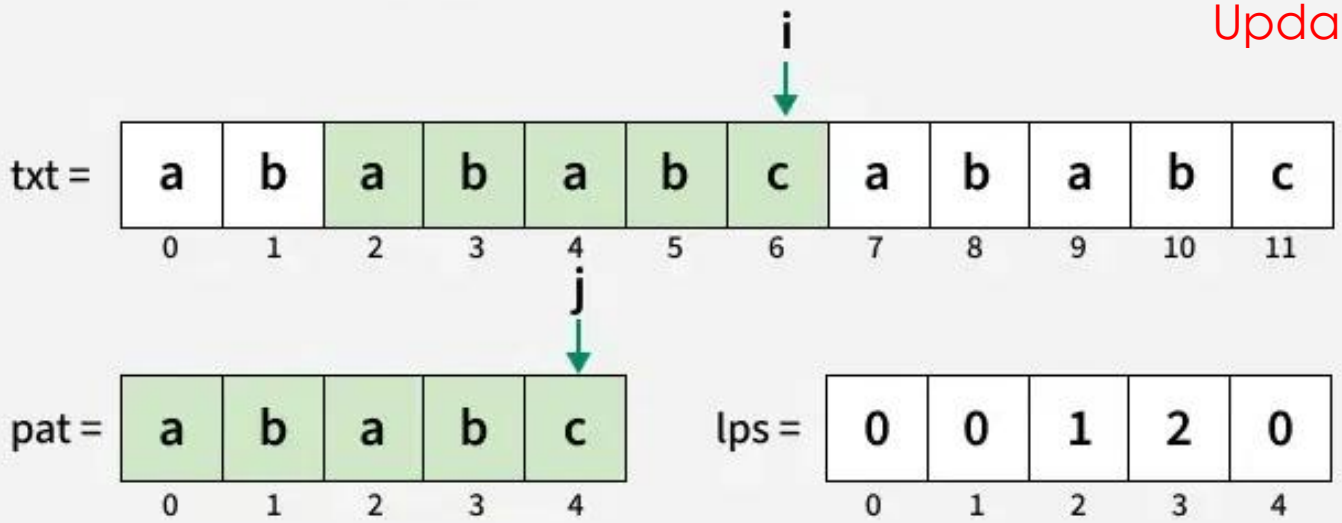
Result = []

KMP Algorithm for Pattern Searching

09 Step

Since txt[i] and pat[j] match, increment both i and j.
Entire pattern has been traversed add the starting index (i-j+1) in the result
And update j to lps[j-1] = 0.

When the pattern is matched
Update j to the last value in LPS

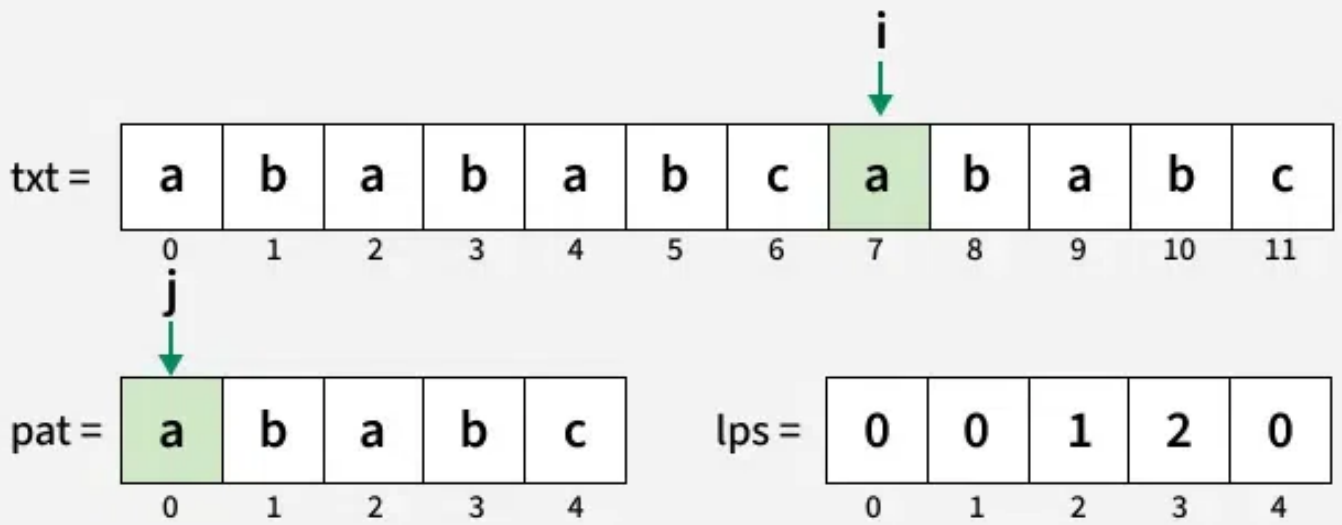


Result = [2]

KMP Algorithm for Pattern Searching

10 Step

Since txt[i] and pat[j] match, increment both i and j.

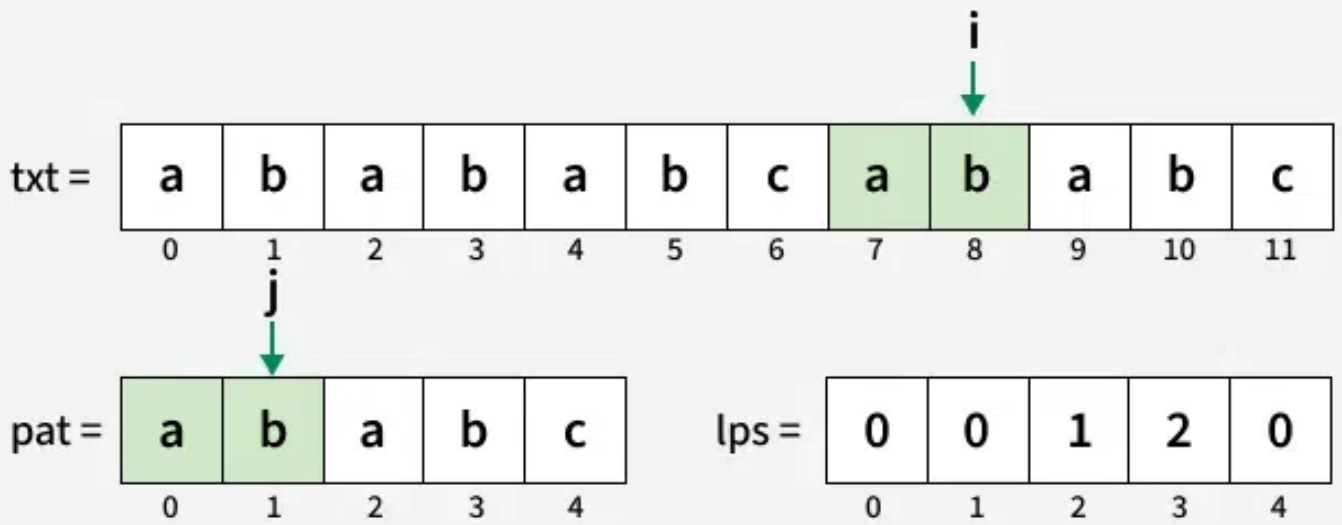


Result = [2]

KMP Algorithm for Pattern Searching

11 Step

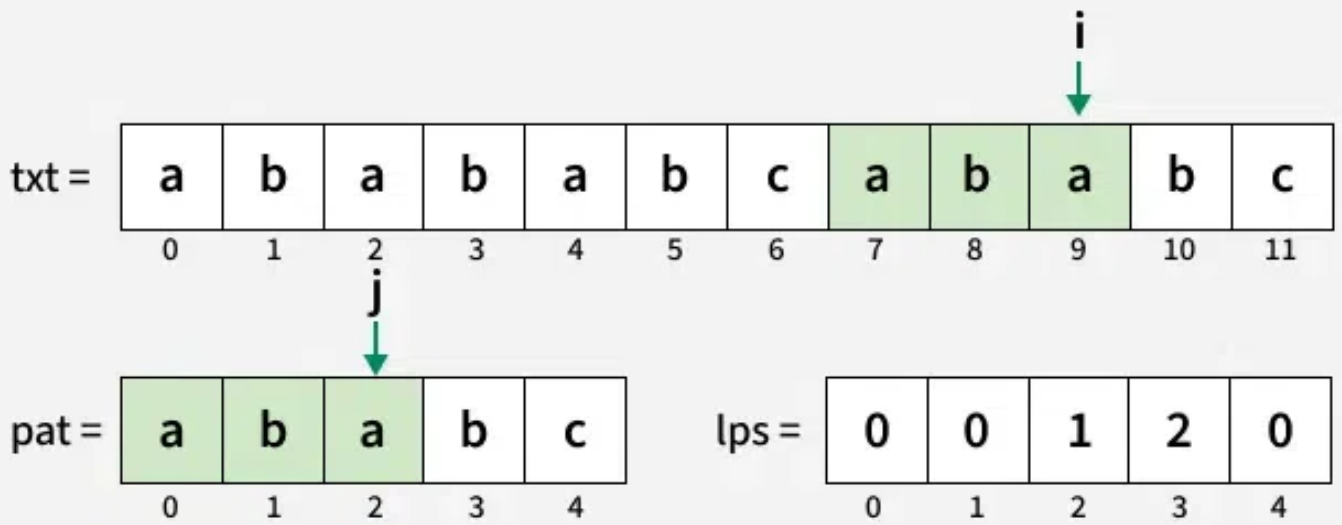
Since txt[i] and pat[j] match, increment both i and j.



Result = [2]

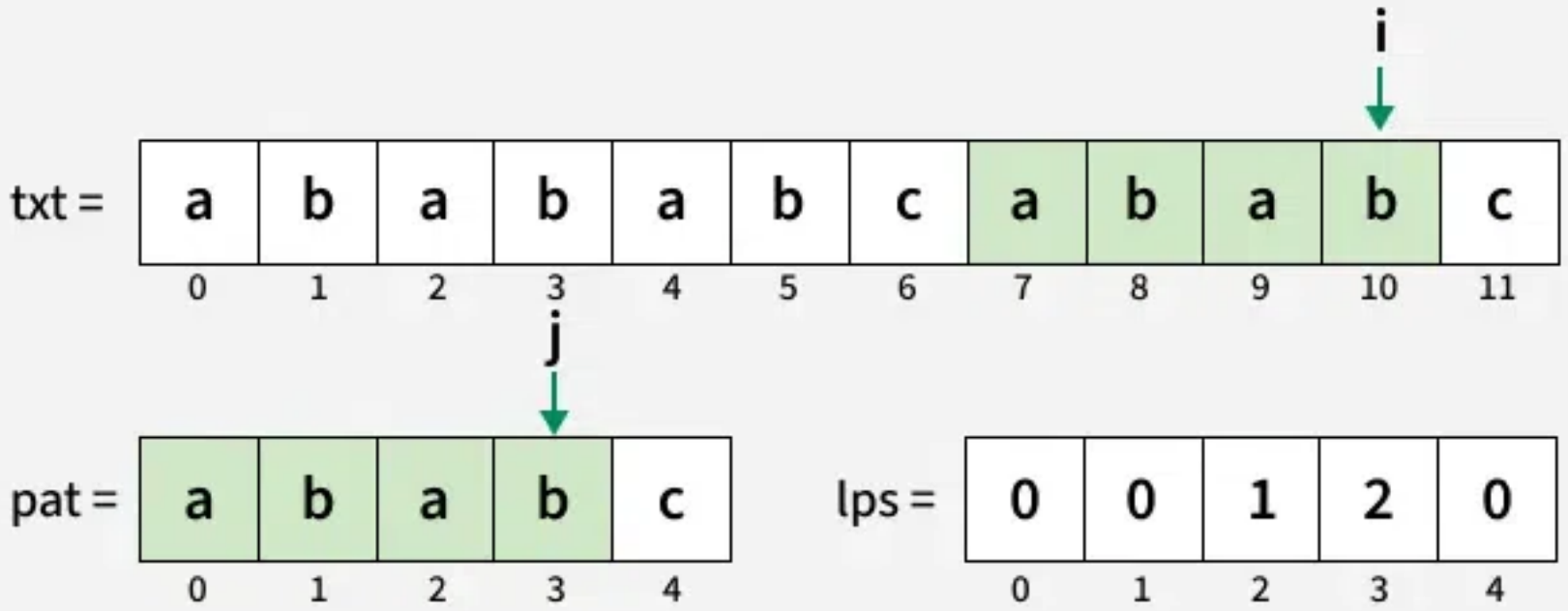
KMP Algorithm for Pattern Searching

12 Step | Since txt[i] and pat[j] match, increment both i and j.



Result = [2]

13 Step | Since txt[i] and pat[j] match, increment both i and j.

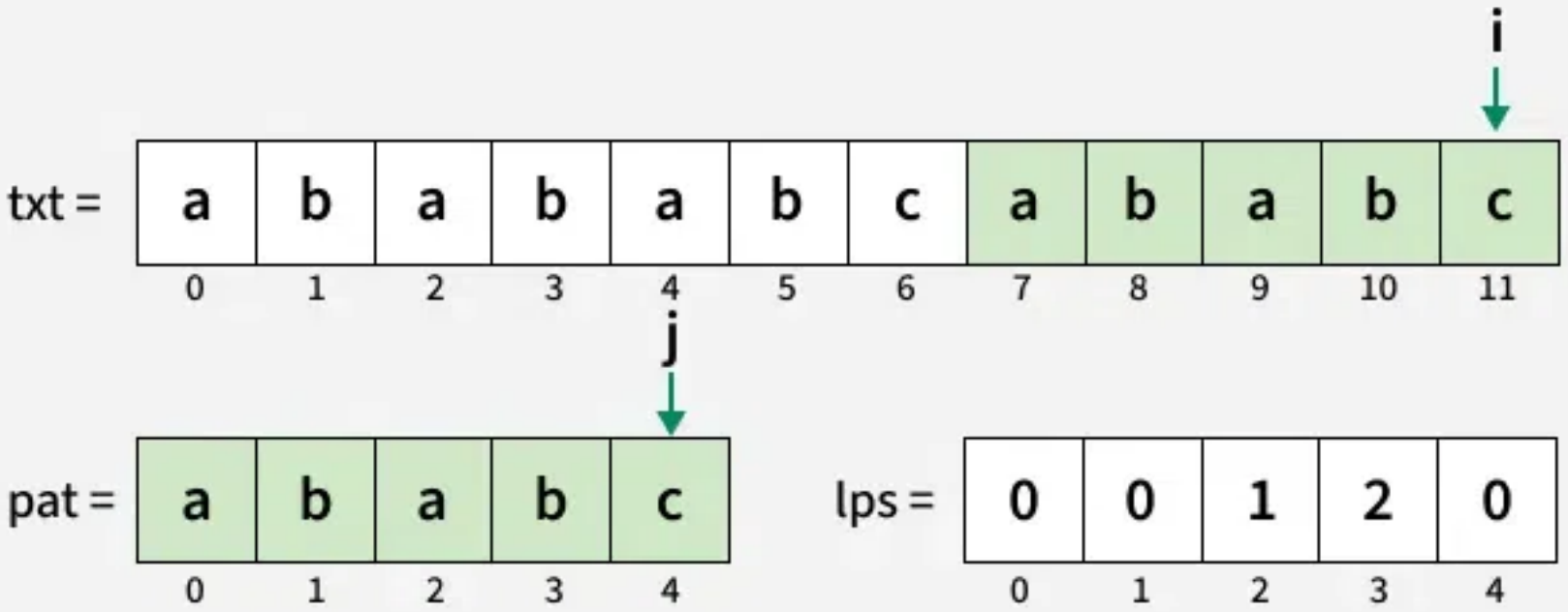


Result = [2]

KMP Algorithm for Pattern Searching

14 Step

Since txt[i] and pat[j] match, increment both i and j.
Entire pattern has been traversed add the starting index (i-j+1) in the result.



Result = [2, 7]

KMP Algorithm for Pattern Searching